

ELEC 378 Spring 2026

Final Project Report

Austin Yilmaz, João de Lima Vana, Rishabh Rengarajan

Contents

1	Introduction and Team information	3
2	Data exploration	3
2.1	General appearance of the dataset	3
2.2	Dataset split for training and validation	5
3	Preprocessing of Data and Design Choices	5
3.1	Preprocessing for the CNN Classifier	5
3.2	Preprocessing for Kernel SVM	6
3.3	Background removal	7
3.4	Brief comparison of the used preprocessing pipelines	7
3.4.1	Feature extraction	7
3.4.2	Data augmentation	7
3.4.3	Local spatial information	7
3.4.4	Normalization	9
3.4.5	Dimensionality reduction	9
4	Models Considered	9
4.1	ResNet CNN	9
4.1.1	Why we chose to try it	9
4.1.2	Algorithm/Workflow	9
4.1.3	Design choices	10
4.1.4	Course concepts used	11
4.1.5	Loss function of the model	11
4.2	Model 2: Kernel SVM	11
4.2.1	Why we chose to try it	11
4.2.2	Algorithm/Workflow	11
4.2.3	Design choices	12
4.2.4	Course concepts used	12
4.3	Model 3: SIFT + KNN	13
5	Training Procedures	13
5.1	Model 1: ResNet CNN (with PyTorch Lightning)	13
5.2	Model 2: HOG + RBF kernel SVM (svmkernel_model.py)	14
5.3	Model 3: KNN + SIFT (sift_knn_classifier.py)	14
6	Quantitative Comparison of the Models	15
6.1	Model 1: Neural Network	15
7	Overfitting and Generalization Analysis	15
7.1	Overfitting in Neural Network Model	15
7.1.1	Evidence of overfitting	15
7.1.2	What may have caused it	16
7.1.3	How we tried to overcome it	16
7.2	Underfitting in whatever	16
8	Pipeline for the Final Submission	16
8.1	Preprocessing of the image	16
8.2	Final model choice and its properties	16
8.3	How the image trains the model	16
8.4	How the model classifies the image	16
8.5	Flowchart	17

9 “Post-Processing”	17
10 Results and Conclusion	18
10.1 Results of the project - what we learned	18
10.2 Comments on the final model's performance	18
10.3 What surprised us	18
10.4 Future steps	18
11 Code	18

1 Introduction and Team information

This report presents the challenges faced, how we overcame them, and our insights on the process for the final project of the ELEC378 course. Our task was to create a multiclass image classifier to classify butterfly and moth species and evaluate this classifier's accuracy on a Kaggle contest. We pursued three model architectures—a ResNet convolutional neural network, a kernel SVM, and a custom SIFT + K-Nearest Neighbors classifier approach—and made various design choices that will be explained in detail below.

Our project team is named **The Front Row**; below is listed the team members and their contributions to the final project:

1. **Austin Yilmaz (ay87):**

- (a) Preparing the sectioning and outline for the project report
- (b) Coding a Convolutional Neural Network classifier training and evaluation pipeline through Python
- (c) Coding an SVM-based classifier training and evaluation pipeline through Python
- (d) Writing the model/workflow-related parts of the project report

2. **João de Lima Vana (jd185):**

- (a) Responsible for training and testing CNN locally
- (b) Developed a model for SIFT with KNN model for the non neural network model
- (c) Developed pre-processing techniques for data (evaluated different packages for data processing: e.g. background removal)
- (d) Developed “post-processing” technique

3. **Rishabh Rengarajan (rr150):**

- (a) Set up Github repo for collaborative work
- (b) Did initial (more naive) work on a SVM-based classifier model
- (c) Responsible for training/testing CNN models locally and on Kaggle, and for tuning hyperparameters
- (d) Implemented `pytorch-lightning` into the CNN classifier for easier logging/streamlined code/faster iteration testing

2 Data exploration

2.1 General appearance of the dataset

At a high level, the dataset consisted of 12594 labeled images for the training set, and 1000 unlabeled images for the testing set. Both datasets contained images of 100 different classes, one class for each image (these classes being the particular species). In the training set, each class was associated with an average of 126 samples, with a minimum of 100 images and a maximum of 187 images per class, as pictured below in figure 1:

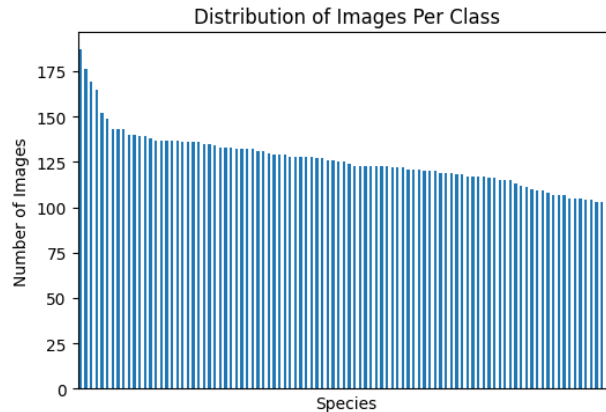


Figure 1: Distribution of species in training dataset

Furthermore, while each image was exactly 224x224x3 pixels (since they were RGB), we noticed a large variation in the information within images. More detailed images had a higher file size while simpler images (for example, white backgrounds) had lower file sizes. The largest pictures contained almost 52kB of data, and the smallest contained about 8kB, while most of the images contained 20 to 30 kB, as shown in the plot below:

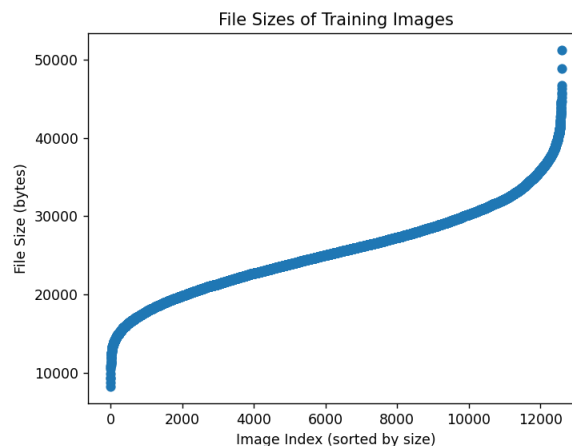


Figure 2: Distribution of image file sizes

In terms of visual variability, we noticed two broad categories of images: insects in a nature scene, or insects in an expository position (typically a white background, fully open wings, and front-facing). Before doing any work, we hypothesized that the exposition-pose images would be generally easier to classify due to the obvious location and clarity of the subject, while natural images could be potentially difficult due to flowers, grass, and other objects in the photograph other than the subject. For example, consider the following examples:

The exposition pose is the easiest one in terms of preprocessing (as our background removal packages excel at removing single colored backgrounds). Meanwhile, the natural ones posed a bigger challenge, as some pictures were overexposed (e.g. test_image 00999), some were on top of flowers (e.g. train_image 000199), which made edge detection harder. Also, we have realized that **color** and **wing shape** were the most drastic differentiators between two species of butterflies, so we incorporated the intuition we gained from those differences into the approaches we used in the preprocessing stage of our data.

the following paragraph is sketchy



(a) An overexposed image, which may be hard to classify (test_000999.jpg).



(b) A saturated image with flowers, which may be hard to classify (train_000199.jpg).



(c) An image in exposition pose (clear subject, white background), likely easier to classify (train_000187.jpg).

2.2 Dataset split for training and validation

A common technique to ensure that we can correctly evaluate our model's performance and tune hyperparameters accordingly is to train the model on a training set and, evaluate it on a validation set. To reduce biases caused by the particular choice of the training/validation split, we also used k-fold cross validation (with $k=5$), in which we repeat the training/validation process 5 times with different splits and average the results to get a more stable estimate of our model's performance. We chose $k=5$ because we used the typical 80/20 train/validation split for all our models (80% of the data for training and 20% for validation) to maximize the number of images we can use for training while still having a good amount of data for validation; with that split, 5 folds would allow us to use all of our data for training and validation across the different folds.

3 Preprocessing of Data and Design Choices

We choose to apply different preprocessing pipelines to our different model architectures, as they have different requirements and are affected differently by the preprocessing steps.

3.1 Preprocessing for the CNN Classifier

All of the transforms we use in our CNN pipeline are located in the `torchvision.transforms` package. Our training pipeline consists of the following steps:

1. `ToImage()`: convert the initial $224 \times 224 \times 3$ image vector to a PyTorch image tensor, which allowed us to utilize CUDA and improve training/prediction speed.
2. `Resize( $224 * 1.1$ )`: upscale the image tensor to about 245 pixels to create some room for randomness and thus help prevent overfitting to exact framings of a picture. (Professor Baraniuk demonstrated similar methods in class.)
3. `RandomCrop(224)`: randomly crop the image to its original size (224×224), to force our model to learn to identify an object "anywhere" in the image, not just in the center.
4. `RandomHorizontalFlip( $p=0.5$ )`: since butterflies/moths are often symmetrically shaped, sometimes horizontally flip the image to prevent the model from having a bias toward left/right facing butterflies.
5. `RandomRotation( $\text{degrees}=10$ )`: a butterfly/moth can also be arbitrarily oriented, so randomly rotate the image anywhere from -10° to 10° to prevent the model from overfitting to the vertical orientations of our butterflies.

6. `RandomAffine(degrees=0, translate=(0.1, 0.1))`: since the butterfly/moth can also be arbitrarily translated in the image, randomly translate the image by up to 10% of its size vertically and horizontally to prevent the model from overfitting to the centered position of our butterflies (for example, in the exposition pose images like figure 3c).
7. `RandAugment(ops=2, magnitude=10)`: beyond simple effects like rotations, translations, and reflections, the `RandAugment` augmentation method utilizes other augmentations that we did not explicitly code our pipeline for. We felt this could especially be useful for further expand the variability of our dataset and thus encourage the model to learn truly useful features.
8. `ToDtype(torch.float32, scale=True)`: convert the pixel values to floats between 0.0 and 1.0, which is a more stable input range for our model to learn from, especially with ReLU activations.
9. `ColorJitter(brightness=0.1, contrast=0.1, saturation=0.1, hue=0.02)`: we wanted our model to tolerate and learn different shades of a colored butterfly/genetic variations/dirt etc., but not too much since butterflies are mainly distinguished by their color, so apply a small random variation to the hue, saturation, saturation, and contrast of the image.
10. `Normalize(mean=(0.5, 0.5, 0.5), std=(0.5, 0.5, 0.5))`: center and normalize pixel values to ensure that the input data works well with ReLU steps in our CNN and that the training produces more stable gradients.

For our validation and test pipeline, we only convert the data to a Pytorch image tensor, ensure the image is 224x224x3, center and normalize, and convert to a float tensor. We do this because all the augmentation steps described above are useful in training the model, but we want consistent/stable validation, so we decided not to apply those preprocessing steps to our validation/test sets.

3.2 Preprocessing for Kernel SVM

review this
for accuracy

Our preprocessing pipeline for the kernel SVM model was quite different and perhaps simpler than the one for the CNN model, partially because the CNN model is more robust to noise and can learn features from raw data, while the SVM model is more sensitive to noise and requires more careful feature engineering. Hence, we applied a simpler and more targeted preprocessing pipeline to our SVM model. In addition, we saw relatively poor initial results from the kernel SVM model compared to the CNN classifier, and we felt that while the SVM model could be potentially improved with more careful feature engineering, the CNN was already performing much better, so we decided to focus our efforts more on improving the CNN model rather than the SVM model. Our SVM preprocessing pipeline consists of the following steps:

1. Resize the image to 64x64x3. We utilize scikit-learn for our SVM implementation, which meant that the SVM training/prediction process is sequential and CPU-heavy, so we wanted to reduce the number of features in our data vectors to improve training/prediction speed. We chose 64x64 because it is a reasonable balance between preserving the features of the image and reducing the number of features.
2. We double the size of our dataset, generating a copy of the images which is horizontally flipped using `numpy.flipplr()`. This exploits the symmetry of butterflies and prevents our model from overfitting to left/right facing orientations.
3. We then used the popular HOG feature descriptor (Histogram of Oriented Gradients) to extract edges, textures, and other shapes of the picture by computing gradients between pixels and comparing their directions. This was because the wing shapes play a huge role in differentiating butterflies, as we observed from our dataset.
4. Unfortunately, HOG throws away color information, which can be hugely important in accurately classifying butterflies and moths. So we used color histograms using `numpy.histogram(bins=32)` to put the red, green, and blue pixel intensities into 32 bins and append 96 features indicating the colors in each image. This way, our model can use color data to classify butterflies more accurately.
5. To avoid brightness/natural noise in colors, we normalized our color histogram values so that the color features are comparable between images.

6. We then append the color histograms to the HOG features.
7. We then normalize and center all of our data vectors, because SVMs (especially with the Gaussian kernel) can be extremely sensitive to disproportionate deviations in features.
8. Finally, since HOG produces very large data vectors, we apply PCA to project all our data vectors onto the first n eigenvectors that capture 95% of the variance so that both training and validation speed improve.

The validation/test pipeline was the exact same except that we did not flip the images, as this would just introduce randomness into the prediction process.

3.3 Background removal

We hypothesized that, due to the extra background information in the natural images (e.g. flowers and grass), a model could potentially perform better if we removed that background information and gave the model only subjects to learn. In other words, we thought that converting all images into “exposition pose” (subject on plain background) would improve our classifier’s accuracy. This segmentation task itself proved to be challenging, and we tried a few methods with OpenCV functions, the most effective of which utilizes the GrabCut image segmentation algorithm using a full-image initial bounding box as well as morphological erosion and dilation operations. After trying different user-chosen parameters with the GrabCut algorithm, This was generally successful at isolating the subjects, but sometimes removed too much from the subject or did not remove enough. Consider the examples of the original image and the background-removed image in Figure 4 below:

This technique improved the accuracy of our KNN+SIFT model a little, but surprisingly led to worse performance in our more complex CNN model and the SVM model . Presumably this occurs because it removes noise from the data, which KNN is more vulnerable to but a CNN can learn.

3.4 Brief comparison of the used preprocessing pipelines

3.4.1 Feature extraction

1. **In our HOG+SVM model**, we deterministically computed edges using HOG and color distributions using histograms. This gave us a fixed representation of each image.
2. **In our CNN mode**, we let the neural network ‘learn’ the features by detecting the edges in the early layers, the textures in the middle layers, the object parts in the deeper layers. This gave us a more adaptive representation of each image, likely superior to HOG+SVM since if that fixed representation loses some information about the butterfly, the HOG and color features alone cannot recover it, while a CNN can learn more about the useful features by introducing layers.

3.4.2 Data augmentation

1. **In our HOG+SVM model**, we only did a horizontal flip on the images. This is because edges do not change much by color/lighting changes and since HOG is a much lower capacity model (compared to neural networks), too much augmentation can drastically reduce model accuracy rather than improving it.
2. **In our CNN model**, we applied a bunch of different random augmentations on our data (as mentioned above), ranging from cropping it to changing the contrasts. This is because even though CNNs learn the features of an image well, they might sometimes learn them *too well*. We need a lot of variation in augmentation to prevent overfitting to the data, especially in a dataset like this where in many of the images the object (butterfly) is relatively in the center of the picture.

3.4.3 Local spatial information

1. **In our HOG+SVM model**, the preprocessing steps (HOG+color histograms) do not really tell us about the locations of a color in an image, which means we lose some spatial information while training. In non-neural-network pipelines, especially a HOG-based SVM, this is inevitable because HOG cannot differentiate

i dont think we ever tried training the SVM with the back-ground re-moved images, maybe we should mention this to be honest

review this statement

review

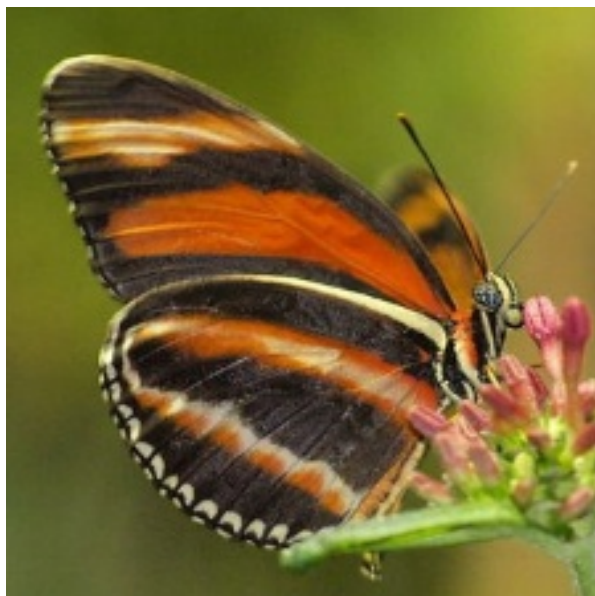
what do u mean by “introducing layers”



(a) test _00003.jpg, original.



(b) test _00003.jpg, cropped; near-perfect segmentation of the subject and background.



(c) test _00081.jpg, original



(d) test _00081.jpg, cropped; failure to fully remove the flowers.

Figure 4: Original images and background-removed images

colors, only shape, while separate preprocessing approaches like color histograms only differentiate color, not shape.

2. **In our CNN model**, we use *convolutions* to propagate the information regarding where particular colors appear in the image. Unlike the SVM approach, this helps us pinpoint how each part of a subject is colored and we can use that to differentiate it from other images.

3.4.4 Normalization

1. **In our HOG+SVM model**, we normalize the features of our data *after* we are done with extracting the features. This is because HOG representations make the RBF kernel SVM extremely vulnerable to small disproportionate variations in features.
2. **In our CNN model**, we normalize the pixels of our data *before* we start learning the features. This is because the CNN needs a stable input range to learn the features and is much more stable to small variations compared to other approaches like the SVM.

3.4.5 Dimensionality reduction

1. **In our HOG+SVM model**, we apply PCA on our data vectors to capture 0.95 of the variance, because training and testing the SVM is a sequential, CPU-heavy process.
2. **In our CNN model**, we do not externally apply any dimensionality reduction but let the layers of our neural network handle it. The pooling, convolutions, and compression inside the hidden layers do the 'dimensionality reduction' for us. Moreover, we were able to parallelize the CNN training, such that it remains efficient with more features.

4 Models Considered

4.1 ResNet CNN

4.1.1 Why we chose to try it

We have read some articles online that the maximum image classification accuracies achieved in diverse datasets like Caltech 101 were around 70–80% without the use of convolutional neural networks (CNNs) [5], while CNNs broke this barrier, attaining 90–99% accuracy with ease [1, 4]. Also, the final project description explicitly states that one of our model choices shouldn't be a neural network, which suggested to us that a neural network might perform quite well. In addition, a CNN is a well-known architecture for approaching image classification problems like MNIST digit recognition or the ImageNet dataset, so we expected it to perform well on our butterfly classification problem as well.

4.1.2 Algorithm/Workflow

Like a plain neural network, the CNN passes images through a series of layers before ultimately predicting class probabilities with a softmax/multinomial logistic regression output layer. The advantage of the CNN is that it uses matrix convolution rather than explicit vector/array multiplication to significantly reduce the number of model parameters. We further decided to use **residual skip connections**, which allow input data to “skip” layers and be directly added to the activation output of a future layer. This approach mitigates the issue of gradients vanishing in backpropagation and allows us to significantly deepen the network, which is a desirable attribute. The workflow is as follows:

1. We pass preprocessed images to the first layer.
2. The first layer, we first perform feature extraction on the input using 2-dimensional convolutions to learn the edges and color gradients.
3. We then downsample the data using max-pooling so that we only keep the strongest indicators/features.

add image of
resnet archi-
tecture?

review these
steps

4. Then, we pass them to the middle hidden layers, which build up increasingly complex patterns from the features using an arbitrary/tuneable variety of approaches where we will discuss in the *How Each Model Was Trained* section.
5. After that, we perform a spatial compression using average-pooling, which removes the spatial dimension to only leave the information about 'how strongly each feature was present in the data'.
6. Finally, we flatten our data to a vector that is the size of the number of our classes, similarly to how one would do 'logistic regression' in the final layer of our network.
7. Now that we are ready to score our vector, we optimize over the cross-entropy loss function we learned in class, with a good optimization algorithm (adamW, for example).
8. We then backpropagate the gradients we get from our optimization algorithm, and update the weights of our model.
9. We repeat this process until our loss converges, or until we hit the maximum number of epochs we set.

4.1.3 Design choices

review these

1. **Amount of feature complexity through layers:** We want to gradually increase feature complexity through the layers of our neural network, so in the first layers, we want to learn simpler things like edges and color; but as we go deeper in the network, we also want to learn more specific things like the markings on a butterfly. So, we need to gradually increase feature sizes and complexity of the operations we perform in the layers and residual blocks. (for example, start with just linear layer + relu, and continue with multiple layers of relu + 1-norm normalization)
2. **How much to reduce image size as we process it:** We want easier to analyze, smaller representations of an image that preserves what is present in the image correctly. But we also do not want to lose the actual features of an image that differentiate it from the others. So, we need to choose the step by step scaling accordingly.
3. **The number of residual blocks and layers we use:** We want a model that both accurately detects objects (butterflies) and has high performance in training, validation and testing, also speed-wise. Therefore, we need to be careful about adjusting how many features each residual block introduces into the data, and how many of the layers/blocks we have. Too many layers and residual blocks can impact performance while introducing marginal accuracy/maybe causing overfitting, but too few can cause the model to underfit or become unstable.
4. **Learning rate: scheduled or constant?** A learning rate too large can cause the model to become very unstable or overshoot the optimum, but one too slow can negatively impact the learning performance of our model. Hence, we should decide on what the initial learning rate is, and if we should decay the learning rate or not.
5. **How to decay the model weights over time:** Too large model weights can cause the model to overfit, and too much decay in the weights can cause the model to underfit. We also need to implement a schedule on decaying the weights of our model to keep it consistent.
6. **What to choose the batch size:** Since the optimization methods we use are usually based on batch optimization (like adamW), we need to choose a batch size that is enough to produce a stable training process, but one that is not too large so that the training stays fast.
7. **What to choose the loss function and optimizer:** Like in many other problems, the cross-entropy loss function is generally unmatched in its success here. So, we do not regard it as that flexible of a choice but still wanted to include it here. Coming to the optimization algorithm, we need an optimization algorithm that both incorporates fast early learning, and later fine-tuning. Moreover, it would help us if our optimizer decayed the weights/scheduled the learning for us. (Yes, we are hinting at adamW, adaptive moment estimation with weight decay)

8. **The preprocessing steps before the data is input:** This is in fact the most important part as CNNs depend heavily on their input, just like most classification models. We have already discussed these choices and the steps we used in the previous section.

4.1.4 Course concepts used

1. **Supervised learning/Classification:** The ResNet CNN we trained is a supervised learning model where inputs are images (data) with labels and the outputs are a discrete number (100) of classes. Hence, this is a mainstream classification problem which we have learned in class to approach with methods like logistic regression, KNN, SVMs, and of course, neural networks.
2. **Feature engineering and learning:** Our CNN learns the features of the data by applying convolution layers and residual blocks.
3. **Optimization:** We have a convex loss function that is called cross-entropy (log-loss), and we use an optimization algorithm based on *Gradient Descent* that is called adamW.
4. **Linear algebra background:** Since we are working with matrices, tensor operations, convolutional layers, max-pooling, and backpropagation of gradients, our work is based in ideas from vector calculus and linear algebra.

this seems vague or useless to say?

4.1.5 Loss function of the model

Our CNN uses cross-entropy loss and uses the AdamW optimizer in gradient descent. The cross-entropy loss is derived from Maximum Likelihood Estimation (MLE) of logistic regression, and it has two terms, where one activates and the other deactivates. It is also convex, and hence optimizable by gradient descent-based algorithms. AdamW is a fairly new algorithm (2017, [3]) that uses “adaptive moment estimation” and “weight decaying”, in which the “moments” are closely related to gradients in gradient descent; and weight decaying is just a smart way of steering the model away from underfitting to more previous data in the training procedure.

revise discussion of AdamW

4.2 Model 2: Kernel SVM

4.2.1 Why we chose to try it

We chose the Kernel SVM as the required “non-neural-network based model” for the project (in addition, we were required to implement one in Homework 7 of the course). We read some articles online that suggested with the right training regimen and preprocessing (HOG), kernel SVMs can achieve accuracies as high as 70% in multi-class classification problems similar to our dataset (like Caltech 101), so it was worth trying to see how well it would perform on our butterfly classification problem.

citation needed

4.2.2 Algorithm/Workflow

Support vector machines (SVMs) learn a hyperplane to separate the data points. Moreover, to increase generalizability, they try to maximize the margin of the hyperplane by learning a “hyper-slab” and maximizing the width of that slab. This ensures that the classification model is more stable for the future data to come, even if it trades off some accuracy for the training data while doing so. Our data is almost always not linearly separable in the original low number of dimensions, so we use the Gaussian kernel to implicitly transform the data into infinite dimensions (which is generally linearly separable given a finite amount of data, guaranteed by the Stone-Weierstrass theorem discussed in lecture) while using the kernel trick to optimize our loss function and train the SVM efficiently. The workflow is as follows:

you said 'regularize' which I don't understand what that means

1. Preprocess the data as described in the previous section.
2. We use the Gaussian kernel to map our data to “infinite” dimensions.
3. Using the kernel trick, minimize our loss function, which adds the hinge loss and the margin maximization,

to learn the parameters of our SVM model. That loss function looks like the following:

$$\mathcal{L}(\alpha, b) = \sum_{i=1}^n (1 - y_i(\alpha^T \mathbf{X} \mathbf{x}_i + b))_+ + \lambda \|\mathbf{X}^T \alpha\|$$

where α and b are the parameters of our SVM model, y_i is the label of the i -th data point, $\mathbf{X}$ is the matrix of our data points, and λ is the regularization parameter that controls how much we want to maximize the margin vs. minimize the hinge loss.

4. While training, depending on $\lambda = 1/C$, we allow some of the points be misclassified or lie within the margin, for which we call them our **support vectors**.
5. We then use the support vectors to implement our decision function, in which the decision function looks like as follows:

$$f(\hat{\mathbf{x}}) = \sum_{i \in SV} \alpha_i y_i K(\hat{\mathbf{x}}_i, \mathbf{x}) + b$$

6. We predict the class of the validation/test set. i.e.:

$$\hat{y} = \operatorname{argmax}_k f_k(\hat{\mathbf{x}})$$

4.2.3 Design choices

1. **Kernel choice:** We need a kernel that has enough dimensionality to get us a linearly separable transformation, but also one that prevents overfitting and is stable. The gaussian kernel is known to have both of those features. By contrast, the sigmoid (tanh) kernel is known to be unstable even though it also maps the data to "infinite" dimensions; and the polynomial kernels are known to overfit/underfit easily to complex datasets.
2. **Regularization parameter C :** Do we want higher bias, lower variance (higher C) in our hyper-slab, or vice versa (lower C)? This can drastically change our prediction accuracy, and we adjusted this hyperparameter to get higher prediction accuracy/lower overfitting levels.
3. **Gaussian exponentiation scaler γ :** If we increase this value, even the slightest spatial local adjustments between data can affect our classifier, which can lead to overfitting. However, if we decrease this value, our classifier may underfit. We also need to pay attention to this hyperparameter to have a good classifier.
4. As with the CNN model, the preprocessing of the data is the most important of the design choices we have made. Most of the work we did in training this classifier was considering how best to preprocess the data (finally deciding to HOG+PCA). If the input data does not provide a good representation of the images we want to classify, the SVM will perform poorly no matter how well we tune the hyperparameters of the SVM.

4.2.4 Course concepts used

1. **Supervised learning/Classification:** Similarly to the CNN model, we do supervised learning here, and the problem is a classification problem.
2. **Optimization/SGD:** We solve a convex optimization problem by minimizing hinge-loss while maximizing margin in the SVM. We are using a variant of (stochastic) gradient descent to solve this problem.
3. **Unsupervised learning/PCA:** This is only used in the preprocessing step to make our training process smoother and faster.
4. **Feature engineering:** While this is also only used in the preprocessing step, our decision to utilize HOG and color histograms is crucial to our model's performance and we need a good manual design of features to accurately represent our data when using an SVM.
5. **Kernel methods/Kernel trick:** To make our data vectors linearly separable, we nonlinearly transform the data using the RBF kernel, and practically optimize the SVM using the kernel trick.

I added "lie within margin", i believe these are also support vectors, but review this statement

what is SV? support vectors?

what is k here?

cite?

6. **Linear algebra background:** We are also working with matrices and vector operations here both in the preprocessing of the data and the training of the model. We also use gradients in the optimization for this model, so our work is still based in vector calculus and linear algebra.

4.3 Model 3: SIFT + KNN

When we started looking for filters for our image processing step, we noticed that the SIFT feature descriptor (introduced in class) reduced each image from several thousand features ($224^2 \cdot 3 = 150528$ pixels per image; even if resized: $64^2 \cdot 3 = 12288$ pixels per image) down to 196. On the other hand, the K-Nearest Neighbors classification algorithm is known for accuracy but also for its high time complexity, so we hypothesized that we could reduce the dataset's feature size using SIFT and then classify with KNN.

Furthermore, as the points of interest are scale and rotation invariant, this improves KNN ability to deal with rotated pictures (which was a severe issue as it changes almost all the pixels around).

Ultimately, after limited testing (as aforementioned, we invested the bulk of our time into improving our CNN model as it showed good results early on), this model achieved at best 42% test accuracy. We considered other ways to improve this model. However, we ran into a few issues, the most important one being that SIFT does not account for any color information, an attribute likely essential for classifying butterflies. Moreover, when we started looking into other possible filters, almost all of them increased the image complexity—which made KNN too inefficient for classification. Thus, given that the performance (at best around 40%) was considerably worse than the our simple initial implementation of SVM, we decided not to proceed further with this architecture.

5 Training Procedures

We describe the training and validation process of our models as implemented in our Python code in this section (PyTorch Lightning notebook `kfold_clf.ipynb`, `svkernel_model.py`, and `sift_knn_classifier.py`). Preprocessing and data augmentation are already summarized in the previous sections; hence, here we focus on data splits, optimization, schedules, regularization, and how we trained each classifier.

5.1 Model 1: ResNet CNN (with PyTorch Lightning)

We used PyTorch Lightning to train our CNN classifier, as the library allowed us to streamline our code, as well as quickly iterate training the model with different hyperparameters and maintain comprehensive logging of the training process. The model architecture is based on ResNet-18, which has 18 layers with three structural changes compared to the ImageNet: the first convolution uses stride of 1 and padding of 1 (we downsample later compared to the usual ResNet architectures); the max-pooling stage is replaced by an identity mapping to preserve more fine-grained information; and the original fully connected layer is replaced by a two-part head dropout block with probability 0.4 followed by a linear mapping from 512 features to 100 classes (the output number of classes). The model applies dropout only at the head and not inside the residual blocks.

1. **Data partitioning.** We use scikit-learn's `StratifiedKFold` with shuffling (seed 42 for reproducibility), and `n_splits=6` for 6-fold cross validation (even though we mentioned 5-fold above, averaging across our 6-fold results proved marginally superior in our experiments). This way, we train and test our classifier by averaging out the parameters learned from the whole dataset, yielding a more stable training and test regimen.
2. **Mini-batch training.** For each fold in our training and validation process, `ImageDataset` maps CSV rows to integer labels via a fixed alphabetical ordering of class names. The training dataloader uses a batch size of 32, random shuffling, `num_workers=4` for faster training/prediction, and `pin_memory=True` for increased performance. Validation and test prediction loaders use the non-augmented preprocessing pipeline aforementioned since they should make predictions on the raw data; and the test `DataLoader` uses a batch size of 1 to make predictions sequentially.
3. **Class imbalance in the loss.** Before we fit a fold, we compute class frequencies on the fold's training frame. We set the class-weights proportional to the inverse frequency (total count divided by class count,

what does training fr-mae mean?

scaled by the number of classes) and pass them into `torch.nn.CrossEntropyLoss` as tensor weights. This reweights the softmax cross-entropy so we balance exploration and exploitation by making species with less number of samples contribute relatively more to the gradients.

what does this mean?

4. **Loss, label smoothing, and optimization.** The training configuration sets label smoothing to 0.1 combined with the weighted cross-entropy approach we discussed above. This replaces one-hot truth label vectors by smoother versions, ideally improving the model's generalizability and accuracy. Our optimizer is AdamW with initial learning rate 3×10^{-3} and weight decay 10^{-2} . A cosine annealing scheduler (`CosineAnnealingLR`) runs through the maximum number of epochs with `T_max = 100`, stepping once per epoch. We also enable mixed-precision training when supported (16-mixed) for increased performance and better memory complexity; and we also set `torch.set_float32_matmul_precision('medium')` for the Tensor Core behavior on the utilized NVIDIA GPUs to help speed up the training process.
5. **Stopping, checkpoints, and logging.** We use early stopping in our training process based on the validation accuracy (we could also have used the validation loss) with a patience of 12 epochs (i.e. wait 12 epochs of no improvement) to prevent overfitting. We continually store the top three checkpoints by validation accuracy, as well as saving the last checkpoint so we can pause and resume training as needed. We also comprehensively log the training and validation loss/accuracy and optimizer learning rate/momentum directory for each fold of the training process and use a learning rate monitor and a progress bar for easier analysis of checkpoint convergence.
6. **Ensembling across folds.** After training all folds, we load the checkpoints and either average the logits across folds or take a per-sample majority vote among the argmax predictions from each fold (averaging proved more successful in practice). After that, we map the indices back to species names for submission. We use this approach to exploit the diversity of our dataset and have a more stable classifier since it has been empirically shown in studies that ensembling can help increase accuracy while also improving overfitting [2].

Explain why use AdamW, move details from below up here

Discuss how we also tried CosineAnnealingWarmRestarts and ReduceLROnPlateau but the most significant change to our accuracy came from using a small initial learning rate and our ResNet architecture

5.2 Model 2: HOG + RBF kernel SVM (`svmkernel_model.py`)

Unlike our ResNet CNN model, we do not use k -fold cross validation in this model. However, if we wanted to further improve this model, we would definitely implement it. We still map the labels to integers using a scikit-learn `LabelEncoder`.

We use scikit-learn's implementation of the SVM classifier with the Gaussian kernel (seed 42), and set the hyperparameters to the implementation's defaults, $C = 1$ and γ to the "scale" heuristic. We fit the model on our full PCA-reduced, augmented training set in one batch solve.

maybe mention that this was because we did not spend much time on improving this model after seeing the initial results of the CNN

5.3 Model 3: KNN + SIFT (`sift_knn_classifier.py`)

Our pipeline for this model is light and uses mostly already implemented packages, as this is more of an experiment because Prof. Baraniuk stated in class that K-NN performs well in general even though the complexity is high. We use OpenCV's SIFT implementation after grayscaling the images in the training dataset. We then standardize the variable length SIFT output to vectors of fixed length and concatenate the mean and standard deviation of all 128 rows to the data, giving 256 features per image. If no keypoints or descriptors are returned, we substitute that with a zero vector - but this rarely happens in our dataset from our observations after training the model.

explain what this does

1. **Training subset and k -NN.** To reduce runtime, we only "sample" every fifth row of `train.csv`. We skip (`idx % sample_rate != 0` with `sample_rate=5`), so, roughly 20% of labeled data participate in fitting. This is to increase performance since as we mentioned, KNN is a slow classifier with high time complexity. We standardize the features with scikit-learn's `StandardScaler` on that subset. We use scikit-learn's `KNeighborsClassifier` with the default $K = 5$ neighbors and Euclidean distance metric. We do not perform an explicitly training/test split on this dataset to keep complexity of the model low, and performance high.

6 Quantitative Comparison of the Models

6.1 Model 1: Neural Network

We obtained the following accuracies for our best (highest Kaggle test accuracy) results with each model architecture:

Table 1: Model Performance Comparison

Model	Train Accuracy (%)	Validation Accuracy (%)	Kaggle (private leaderboard) Test Accuracy (%)
CNN	98.6	92.9	99.8
SVM	96.1	56.1	63.4
KNN	48.5	29.8	37.0

Note that the train and validation accuracy reported for the CNN is the average accuracy across all 6 folds in the best result we obtained, and the test accuracy is the one from our best submission on Kaggle. For the SVM and KNN models, we only have one training/validation accuracy since we did not have the time to implement k -fold cross validation for those models.

Evidently, the CNN performed far better than the other two models, which is consistent with our expectations given the flexibility of the architecture and the fact that the ResNet CNN architecture is a state-of-the-art approach for image classification problems. The SVM performed decently, but it was still far behind the CNN, which is likely because of the limitations of the HOG features and the fact that SVMs are not as powerful as CNNs in learning complex patterns in data. The KNN model performed the worst, likely due to its sensitivity to noise in the data, as well as the fact that SIFT does not capture color information which is likely essential for classifying butterflies correctly.

Surprisingly, in all of these results, the train accuracy is far higher than the validation accuracy. This is a sign of overfitting, which is expected given the complexity of these models and the fact that our data is relatively small (12000 images) compared to the number of parameters in our models (especially the CNN). However, the test accuracy of the CNN is much higher than the validation accuracy, which is a sign that that model is actually generalizing well to unseen data, and that our validation set was not representative of the test set. This is likely because of the fact that we used k -fold cross validation, which allowed us to train on more data and have a more stable model.

Notably, the SVM experiences a 40% drop from train to validation accuracy, which is a sign of severe overfitting.

7 Overfitting and Generalization Analysis

TODO: Describe the instances where your models overfit/underfit, by first stating which model it was, and elaborate on what caused it. Then discuss strategies you resorted to, to overcome that over/underfitting. You might find it useful to explain overfitting/underfitting using the validation/test accuracy discrepancies of the model stated above.

7.1 Overfitting in Neural Network Model

7.1.1 Evidence of overfitting

The main evidence of overfitting in our model, was that we had a training accuracy of over 99%, and a validation accuracy of 93% (our final submission accuracy was somewhat higher, but we achieved that by using a voting model across our different data splits). So, this is the main trait of overfitting, improved performance on training set, which did not improve the actual model performance.

7.1.2 What may have caused it

As we have learned in class, because neural networks have such exceptionally large number of weights to classify. Thus, as expected, it manages to reduce the training data error significantly, even though it does not translate into actual improvements in the accuracy.

7.1.3 How we tried to overcome it

TODO: Elaborate on what methods you used on mitigating the overfitting, and how well it mitigated it. If you were unable to improve, relate it to model complexity and task difficulty.

We reduced overfitting by doing early stop, using weight decay (i.e. a ridge penalty), and using dropout layers.

7.2 Underfitting in whatever

same argument.

8 Pipeline for the Final Submission

Our best and final submission is the ResNet CNN model. Here we describe the exact pipeline involved in the final submission. First, we preprocess the data as described in Section 3.1. **TODO: Explain each process of an image's journey in the final submission pipeline, starting from its preprocessing up until how it's classified with a given label. HIGHLY RECOMMEND TO ADD A FLOWCHART OR A VISUAL DRAW.IO MODEL ETC HERE!**

8.1 Preprocessing of the image

TODO: Briefly explain how a particular image is preprocessed in the final pipeline.

8.2 Final model choice and its properties

TODO: Briefly explain what model you chose for the final pipeline touching on the properties of the image that the model suits well.

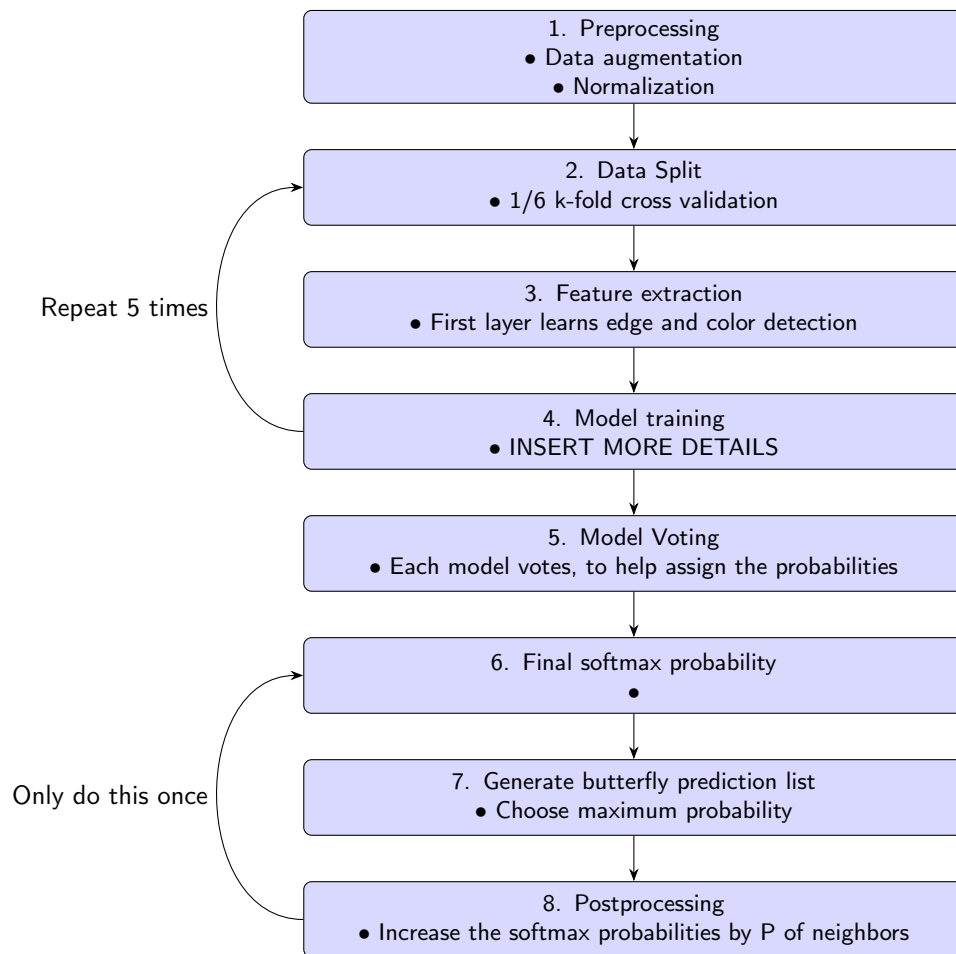
8.3 How the image trains the model

TODO: Briefly explain how the image is used in the training process of the model, touching on exactly how it improves the optimization process

8.4 How the model classifies the image

TODO: Briefly explain how the model processes the image to label it.

8.5 Flowchart



9 “Post-Processing”

(Note for grader: we asked on Piazza whether this is a valid technique, and the instructor response said it was)

After we looked at the test images, we noticed an odd pattern: there was significant cluttering of butterflies in sequential order. This makes sense, considering that if we were to look at a user feeding data into some form of database, they would probably in clusters- as someone might take multiple pictures of the same butterfly. Furthermore, it is expected that the classification would have some natural statistical biases, for example- we might expect that if a past butterfly is from a certain location, then other species in the same region would be more likely. Thus, it makes sense to also account for those patterns when creating a classifier.

Hence, we altered our prediction pipeline. Once we had our predicted list of butterflies, we retroactively scaled the softmax probability vector by a $(1 - 2P)$ and added P (a tunable hyperparameter) to the probability of the neighboring butterflies. With that, we managed to achieve 99.8 % accuracy on the test dataset.

It is worth noting though that this also increases the chance of overfitting to the particular dataset that we were working with, if someone decided to shuffle the data the accuracy would then go down quite a lot; so, it is important to tune the P value.

10 Results and Conclusion

10.1 Results of the project - what we learned

TODO: Reflect on the overall results of the project and what new information you have learned.

10.2 Comments on the final model's performance

TODO: What do you think about how your final model performed in terms of performance and accuracy?

Elaborate. We were reasonably satisfied with the final performance of our models. In particular, the ResNet was quite good at predicting, an

10.3 What surprised us

TODO: List all the things that were unexpected. They can be good, bad, or ugly. We were surprised when looking at some of the misclassified butterflies. While the majority of them had some large visual variation (e.g. the wings were shown from the top, which has a very different visual aspect to the bottom of the butterfly, and looked like another species). However, there some classifications that seemed similar to the rest of the dataset, so we were surprised that they were misclassified (which goes to show, as discussed in class that Neural Networks can be sensitive to small changes, producing unpredictable errors).

10.4 Future steps

11 Code

Our code is available in the GitHub repository https://github.com/rishabhreng/ELEC378_FinalProject, and the main files are `kfold_clf.ipynb` for the ResNet CNN, `svmkernel_model.py` for the SVM, and `sift_knn_classifier.py` for the KNN+SIFT model. There is also a README file that explains how to run the code and reproduce our results.

References

- [1] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *Proceedings of the 26th International Conference on Neural Information Processing Systems - Volume 1*, NIPS'12, pages 1097–1105, Red Hook, NY, USA, 2012. Curran Associates Inc.
- [2] Jinsu Lee, Sang-Kwang Lee, and Seong-Il Yang. An ensemble method of cnn models for object detection. In *2018 International Conference on Information and Communication Technology Convergence (ICTC)*, pages 898–901, 2018. doi: 10.1109/ICTC.2018.8539396.
- [3] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019. URL <https://arxiv.org/abs/1711.05101>.
- [4] M Zabir, N Fazira, Zaidah Ibrahim, and Nurbaity Sabri. Evaluation of pre-trained convolutional neural network models for object recognition. *International Journal of Engineering and Technology*, 7(3.15):95–98, Aug. 2018. doi: 10.14419/ijet.v7i3.15.17509. URL <https://www.sciencepubco.com/index.php/IJET/article/view/17509>.
- [5] M. Zamani F., Jamzad. A feature fusion based localized multiple kernel learning system for real world image classification. In *J Image Video Proc.*, page 78. Springer, 2017.